

e-TCP Surveillance System Project Handbook

Architecture, data flow, technology choices, operations, and maintenance

Generated 08 June 2026

C:\Users\VICTUS\Desktop\Survillance_System

This document summarizes the current project implementation and the design decisions behind it. It is intended for operators, maintainers, and developers who need to understand how data enters the system, how it is stored, how the UI reads it, and how to troubleshoot field deployments.

Executive Summary

The e-TCP Surveillance System is a LAN-hosted vehicle monitoring application for toll/control post operations. It receives ANPR events from CP Plus cameras, stores parsed detections in MySQL, shows live status and logs in a React operator dashboard, and builds TCP movement reports from paired IN/OUT cameras.

The current architecture treats CP Plus ANPR events as the primary detection source. RTSP preview and optional YOLO processing remain available, but heavy OCR/AI work is kept out of the camera request path to avoid lag and missed events.

A key operating rule is raw-first plate handling: the plate shown on the website is the exact value received in the camera JSON. PaddleOCR is only a fallback when the camera plate format is invalid, and any replacement must be strict or confirmed uniquely through the vehicle master database.

Project Goals

Receive vehicle/plate data reliably from multiple CP Plus cameras on the local network.

Map each camera DeviceID/IP to a configured camera id and TCP pair.

Preserve received evidence: raw JSON payload, plate image, vehicle image when available, camera time, receive time, and parsed fields.

Provide an operator UI for live counts, logs, camera previews, reports, receiver events, alerts, and TCP movement status.

Minimize lag by using camera ANPR as the primary source and moving expensive OCR into an asynchronous fallback path.

Technology Stack and Why

Technology	Used For	Why
Python + Flask	API server, camera receiver, JSON parsing	Simple local deployment, good OpenCV/MySQL integration, easy LAN hosting.
Waitress	Production WSGI server for Windows client machines	More reliable than Flask dev server and supports threaded request handling.
React + Vite	Operator UI for dashboard, streams, logs, reports	Fast local development, simple static frontend, component-based UI.
MySQL/MariaDB	Persistent event logs, vehicle master, TCP movement reports	Simple queries, date filtering, dashboard counts, report joins.
CP Plus ANPR API	Primary detection input from cameras.	Camera already detects plates and sends snapshots, reducing live CPU load.
OpenCV	Image decoding, RTSP frame handling, preprocessing	Works well with RTSP/MJPEG and Python image pipelines.
YOLO/Ultralytics	Optional stream/upload vehicle and plate detection	Backend/optional analysis when camera ANPR is not available.
PaddleOCR	Fallback OCR on received plate crops only	Used asynchronously and narrowly so live camera handling does not lag.
aiortc/WebRTC	Optional low-latency camera preview.	Reduces perceived MJPEG lag where WebRTC dependencies are available.

Configuration Model

project_config.json is the main operational control file. It contains the backend host/port/public_url, the 14 camera definitions, RTSP URLs, CP Plus DeviceID/IP mapping keys, and TCP pair definitions.

The important design decision is centralization: when a client machine IP changes, the backend public_url and frontend configuration can be adjusted in one config file instead of editing scattered code.

Each camera entry can contain cp_plus_keys. These keys may be DeviceID values or camera IP addresses. Incoming CP Plus events are mapped to the application camera id by matching DeviceID, remote address, lane, or

channel signals.

Camera and TCP Pair Map

TCP Key	First/IN Camera	Second/OUT Camera
igoo	IGOO TCP to Leh	IGOO TCP to Kiari
kiari	Kiari to Leh	Kiari-CThang
cthang	C/Thang to Kiari	C/Thang to Nyoma
nyoma	Nyoma to C/Thang	Nyoma to Loma
loma	Loma to Nyoma	Loma to Hanle
hanle	Hanle to Loma	Hanle to Tasigang
chushul	Chushul to Tara	Chushul to Parma

Backend Runtime

backend/app.py loads environment variables, sets Paddle-related environment flags, creates the Flask app, enables CORS, checks MySQL, creates or updates tables, imports vehicle details from Excel when MySQL is available, optionally preloads OCR, and finally runs Waitress.

The backend listens on 0.0.0.0 so cameras on the LAN can reach it. The camera itself should use the machine LAN IP from server.public_url, for example http://192.168.2.146:8080.

Waitress is configured with multiple threads so camera POSTs, frontend polling, image serving, and preview requests can coexist better than with the Flask development server.

CP Plus ANPR Event Flow

Primary event pipeline:

```
CP Plus camera
-> POST /NotificationInfo/TollgateInfo
-> Flask notification receiver
-> parse JSON + decode images + duplicate/stale checks
-> save raw received JSON under backend/received
-> save plate/vehicle images under backend/flask_app/static/anpr
-> insert row into MySQL vehicle_logs
-> optional background PaddleOCR fallback for invalid-format plate crops
-> frontend polls APIs for dashboard/logs/receiver/TCP reports
```

Raw Plate Handling

The system intentionally stores and displays the plate text from the camera JSON unchanged. This avoids silently changing evidence and makes operator review honest.

Normalization is still used internally for matching, classification, and master lookup, but the website-facing license value for CP Plus events remains the raw JSON value.

Duplicate detection uses plate/camera/lane/time windows to skip repeated camera resends, while exact payload fingerprinting catches full duplicate events.

PaddleOCR Fallback Logic

```
Camera JSON plate is stored/displayed as received.
If the JSON plate is already valid civil or military format:
-> OCR fallback is not queued.
```

If the format is invalid and the plate color is not civil:

- > background worker runs PaddleOCR on the saved plate crop.
- > strict military-format result can update the DB row.
- > otherwise, it checks exact vehicle_master candidates using visual-confusable characters.
- > if still unresolved, it checks the unique 6-digit military serial in vehicle_master.
- > only one unique DB-confirmed match is accepted.

Classification Rules

Civil plates are identified by known Indian RTO state prefixes and civil plate formats, or by civil plate colors such as White, Yellow, Green, Red, or Blue.

Military plates are identified by strict military format: two leading year digits in the configured range, one military series letter, six serial digits, and one trailing alphabet.

Broad-arrow capture handling remains narrow: if OCR/camera adds a leading or trailing '1' around an otherwise valid military plate, the system may strip that extra marker for internal classification.

The system avoids broad generalization. It does not pad random OCR text into a fake military number and does not use general fuzzy matching.

MySQL Data Model

Table	Contents	Purpose
camera_master	Camera id, display name, TCP group, direction, RTSP url, active status	Used from active project config.json; used for camera metadata.
vehicle_logs	All detections: plate, class, speed, images, camera, time, operational status	Primary operational log and source for dashboard, reports, TCP matching
tcp_movements	Historical IN/OUT movement rows.	Maintained for movement state compatibility; current report also rebuilds f
vehicle_master	Known vehicle plate/unit/type/driver/remarks data.	Imported from datacontrol/veh_details.xlsx and used for unit lookup and C
blacklisted_vehicles	Blacklisted plate and remarks.	Used by Alerts panel and search.

TCP Movement Report Logic

vehicle_logs rows for one TCP pair

- > select both paired cameras for date range
- > sort detections by time
- > first detection for a plate is IN
- > next detection from the opposite camera is OUT
- > unmatched rows remain visible as waiting
- > UNKNOWN/bad OCR rows stay visible instead of disappearing

RTSP Preview, WebRTC, and YOLO Paths

Preview is deliberately separated from YOLO detection. /preview/start_camera starts a lightweight RTSP reader and /camera_feed/ serves MJPEG frames for display.

The WebRTC endpoint /webrtc/offer/ is available where aiortc dependencies are installed. This is useful when MJPEG appears delayed because WebRTC is designed for lower-latency live media.

YOLO RTSP and uploaded-video processing remain available through /yolo/start_camera and /upload_video. These paths are heavier and should be used carefully on CPU-only systems.

Frontend Application

Panel	Purpose
Dashboard	Health, total counts, day/week/month/year charts, camera cards.

Panel	Purpose
Streams	Start RTSP preview, MJPEG/WebRTC preview, snapshots, preview logs.
Upload	Manual video upload and processed video feed.
Logs	Camera-specific detection table with plate and vehicle images.
Reports	Last 7 days and filtered report export.
TCP	IN/OUT movement table by TCP pair, waiting rows, matched rows.
Vehicle Master	Manual vehicle details and Excel import trigger.
Alerts	Blacklist CRUD and license search.
Receiver	Live CP Plus received payload table.
Map	Camera map dots and per-camera popup stats/logs.

Reports and Exports

Dashboard and report endpoints read from `vehicle_logs` using date filters, camera aliases, class ids, and camera ids.

The last-7-days report supports camera and vehicle type filters. PDF export is handled on the backend using ReportLab.

TCP reports are generated from MySQL rather than relying only on the live JSON stream, so reports can be rebuilt after restart and can include historical records.

Vehicle Master and Unit Matching

`datacontrol/veh_details.xlsx` is the source file for master vehicle data. The backend imports it into `vehicle_master` at startup when MySQL is available, and the `datacontrol` script can be run manually for sync.

`vehicle_master` is used to display unit, vehicle type, driver, and remarks on logs and reports. It is also used as a safety confirmation source for OCR fallback on ambiguous military plates.

For military OCR ambiguity, the system first tries approved visual character substitutions against exact valid master plates. If that fails, it can match by the unique six-digit military serial, but only when that serial maps to exactly one valid military plate.

Data Control and Test Utilities

`datacontrol/sync_vehicle_details.py` manually imports Excel vehicle details into MySQL.

`datacontrol/rebuild_anpr_images.py` rebuilds saved ANPR images from received JSON payloads when image extraction logic changes.

`test/test_mil_plate.py` is an interactive rule tester for plate classification.

`test/test_paddle_mil_ocr_speed.py` benchmarks PaddleOCR over received JSON plate crops and writes CSV results.

`test/ocr_image.py` runs PaddleOCR on one image and prints raw OCR output and best cleaned text.

Operational Setup

Backend: create a Python environment, install `backend/requirements.txt`, configure `project_config.json` and `.env`, then run `python backend/app.py` from the backend folder.

Frontend: run `npm install` and `npm run dev` from frontend. The UI defaults to Vite on port 5173 and proxies API calls to the backend.

Camera: enable ANPRAPI, set Platform Server to the backend public URL, set Heartbeat Interface to /NotificationInfo/KeepAlive, keep Need Response enabled, and enable the required upload fields and images.

Firewall: allow inbound TCP for the backend port. Cameras must be able to reach the server public URL from their subnet.

MySQL: ensure the configured user/password/database are valid. The app can start without MySQL, but DB-backed pages remain empty and events are not inserted until MySQL works.

Troubleshooting Guide

No camera posts: verify camera Platform Server, PC IP, firewall rule, same subnet/routing, backend public_url, and that the backend is listening on 0.0.0.0.

KeepAlive works but no TollgateInfo: check CP Plus ANPR upload options, event trigger configuration, and whether the camera is seeing plates in ANPR mode.

Old/stale events: check camera System Time, Time Zone, NTP settings, and CP_PLUS_MAX_EVENT_AGE_SEC.

Duplicate spam: check camera resend behavior and duplicate windows
CP_PLUS_DUPLICATE_PLATE_WINDOW_SEC and exact payload duplicate handling.

No images: verify whether the camera sends VehiclePic, CutoutPic, PlatePic, VehicleBodyCutout, or multipart files. Some payloads include only plate/cutout images.

Lag in preview: lower PREVIEW_STREAM_FPS/resolution/quality, prefer WebRTC where available, avoid running YOLO on all streams, and keep OCR asynchronous.

PaddleOCR install failure: use a supported Python version. Some Paddle packages may not support very new Python releases.

MySQL access denied: confirm MYSQL_USER and MYSQL_PASSWORD. MariaDB/MySQL client may exist even when mysql is not on PATH.

Extension Guide

To add a CP Plus camera, add a camera entry or update cp_plus_keys in project_config.json with its DeviceID and/or IP address.

To add a TCP pair, add two camera definitions and one tcp_pairs entry pointing to the camera ids.

To change the client machine IP, update server.public_url and the camera Platform Server setting.

To tune OCR fallback, use ETCP_EVENT_PLATE_OCR, ETCP_EVENT_PLATE_OCR_QUEUE_MAX, ETCP_EVENT_PLATE_OCR_MASTER_MIN_SCORE, and ETCP_EVENT_PLATE_OCR_MASTER_MAX_CHANGES.

To tune preview, use PREVIEW_STREAM_WIDTH, PREVIEW_STREAM_HEIGHT, PREVIEW_STREAM_JPEG_QUALITY, PREVIEW_STREAM_FPS, and PREVIEW_BUFFER_DROP_FRAMES.

Important Design Decisions

Raw JSON first: the website shows what the camera sent, not what the system guesses.

Asynchronous OCR: PaddleOCR runs after insertion, not in the camera POST critical path.

Exact/unique DB confirmation: ambiguous OCR correction is allowed only when vehicle_master uniquely confirms the result.

Report from DB: TCP report logic is built from vehicle_logs, so it survives backend restart and can be audited historically.

One config file: camera map, TCP pairs, public URL, and RTSP URLs live in project_config.json.

Key API Endpoints

Area	Endpoint(s)	Purpose
Health/config	GET /api/health, /api/app_config, /api/cameras	Status, public URL, camera list, TCP pair config.
CP Plus receiver	POST /NotificationInfo/TollgateInfo	Main camera ANPR event endpoint. Returns plain OK for camera compatibility.
Keep alive	POST/GET /NotificationInfo/KeepAlive	Heartbeat receiver. Saves keepalive JSON and returns OK.
Receiver history	GET /api/notifications/recent	Recent parsed tollgate events shown in the Receiver tab.
Preview	POST /preview/start_camera, GET /camera_feed/<id>	RTSP preview without YOLO, exposed as MJPEG feed.
WebRTC preview	POST /webrtc/offer/<id>	Lower-latency preview path using aiortc when dependencies are available.
YOLO stream	POST /yolo/start_camera, GET /yolo/camera_feed/<id>	Optional RTSP YOLO processing path.
Upload video	POST /upload_video, GET /video_feed	Manual video upload processing and output stream.
Logs	GET /api/camera_logs/<id>	Recent camera detections from MySQL.
TCP report	GET /api/tcp_table/<tcp>	Movement report built from vehicle_logs and TCP pair map.
Reports	GET /api/last_7_days_report, /download_last_7_days_report	File report rows and PDF export.
Vehicle master	GET/POST /api/vehicle_master	Master vehicle details and Excel import.
Alerts	GET/POST /api/blacklist, GET /api/blacklist_alerts	Blacklist management and alert lookup.

Project File Map

Path	Role
backend/app.py	Backend startup, CORS, MySQL setup, OCR preload, Waitress server.
backend/core/common.py	Database schema, insert/update helpers, classification, reports, TCP matching.
backend/flask_app/services/cp_plus.py	CP Plus payload parsing, image extraction, camera mapping.
backend/flask_app/blueprints/notifications.py	TollgateInfo/KeepAlive receiver, duplicate/stale filtering, event persistence.
backend/flask_app/services/plate_ocr_worker.py	Background PaddleOCR fallback for invalid plate crops.
backend/preview_pipeline.py	Low-cost RTSP preview frames.
backend/rtsp_pipeline.py	Optional live YOLO RTSP processing.
backend/video_pipeline.py	Uploaded video processing.
frontend/src/main.jsx	Main React state, polling, tab orchestration.
frontend/src/features/*.jsx	Feature panels for dashboard, streams, logs, TCP, reports, receiver, map, alerts.
datacontrol/*.py	Manual database maintenance and image rebuild utilities.
test/*.py	OCR and plate rule test tools.